

System Design Documents

14. Use Case Diagram

Actors

1. Admin

The Admin is responsible for managing the overall system.

Functions:

- Login to the system
- Manage users
- Create and manage projects
- Assign roles
- View reports
- Monitor system activities

2. QA Engineer

The QA Engineer performs software testing activities.

Functions:

- Login to the system
- View assigned projects
- Add requirements
- Create test cases
- Execute test cases
- Report defects
- Update defect status
- Add comments and attachments
- Generate testing reports

3. Developer

The Developer is responsible for resolving reported defects.

Functions:

- Login to the system
 - View assigned defects
 - Update defect status
 - Add comments
 - Attach fix details
 - Mark defects as resolved
-

Main Use Cases

Authentication

- User Login
- User Logout

Project Management

- Create Project
- Update Project
- Manage Project Members

Requirement Management

- Add Requirement
- Update Requirement
- Link Requirements with Test Cases

Test Case Management

- Create Test Case
- Edit Test Case
- Execute Test Case
- View Test Results

Defect Management

- Report Bug
- Assign Bug
- Update Bug Status
- Comment on Bug
- Close Bug

Reporting

- View Dashboard
 - Generate Reports
-

Use Case Relationship

Admin:

- User Management
- Project Management
- Reports

QA Engineer:

- Requirements
- Test Cases
- Test Execution
- Bug Reporting

Developer:

- Bug Resolution
- Status Updates
- Comments

15. Entity Relationship Diagram

The database design consists of multiple entities that store and manage information related to software testing activities.

Entities

User

Attributes:

- user_id (Primary Key)
- name
- email
- password
- role
- created_date

Relationship:

- A user can create multiple bugs.
 - A user can execute multiple test cases.
 - A user can belong to multiple projects.
-

Project

Attributes:

- project_id (Primary Key)
- project_name
- description
- status
- created_date

Relationship:

- One project contains multiple requirements.
 - One project contains multiple test cases.
 - One project contains multiple bugs.
-

Requirement

Attributes:

- requirement_id (Primary Key)
- project_id (Foreign Key)
- title
- description
- priority
- status

Relationship:

- A requirement belongs to one project.
 - A requirement can have multiple test cases.
-

Test Case

Attributes:

- test_case_id (Primary Key)
- requirement_id (Foreign Key)
- title
- steps
- expected_result
- status

Relationship:

- A test case belongs to a requirement.
 - A test case can have multiple execution results.
-

Test Execution

Attributes:

- execution_id (Primary Key)
- test_case_id (Foreign Key)
- executed_by
- result
- execution_date

Relationship:

- A test case has multiple executions.
-

Bug

Attributes:

- bug_id (Primary Key)
- project_id (Foreign Key)
- title
- description
- severity
- priority
- status
- assigned_to

Relationship:

- A project contains multiple bugs.
 - A bug can have multiple comments.
-

Comment

Attributes:

- comment_id (Primary Key)
- bug_id (Foreign Key)
- user_id (Foreign Key)
- comment_text
- created_date

Relationship:

- A bug contains multiple comments.

16. Class Diagram

The Class Diagram represents the main classes of the QA Management Suite and the relationships between them.

Class: User

Attributes

- userId
- name
- email
- password
- role

Methods

- login()
 - logout()
 - updateProfile()
-

Class: Project

Attributes

- projectId
- projectName
- description
- status

Methods

- createProject()
 - updateProject()
 - addMember()
-

Class: Requirement

Attributes

- requirementId
- title
- description
- priority
- status

Methods

- addRequirement()
 - updateRequirement()
 - deleteRequirement()
-

Class: TestCase

Attributes

- testCaseId
- title
- description
- testSteps
- expectedResult
- status

Methods

- createTestCase()
 - updateTestCase()
 - executeTestCase()
-

Class: TestExecution

Attributes

- executionId
- result
- executionDate
- remarks

Methods

- recordResult()

- generateTestReport()

Class: Bug

Attributes

- bugId
- title
- description
- severity
- priority
- status

Methods

- createBug()
- assignBug()
- updateStatus()
- closeBug()

Class: Comment

Attributes

- commentId
- commentText
- createdAt

Methods

- addComment()
- editComment()

Class Relationships

```
User
|
| creates
|
Project
```

```
|
| contains
|
Requirement
|
| linked with
|
TestCase
|
| generates
|
TestExecution
```

```
Project
|
| contains
|
Bug
|
| has
|
Comment
```

The class structure supports modular development and allows future expansion of the system.

17. Sequence Diagram

Scenario: Reporting a New Bug

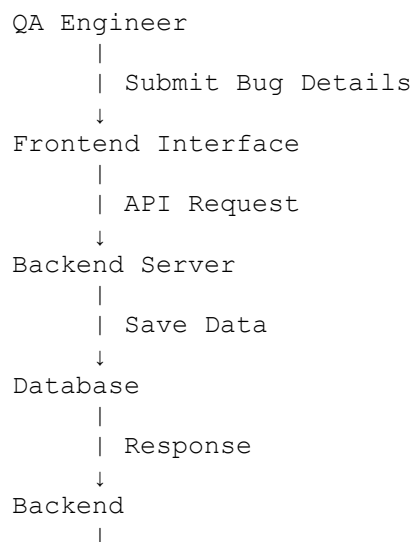
Actors:

- QA Engineer
- Frontend System
- Backend Server
- Database

Sequence Flow:

1. QA Engineer logs into the system.
2. User opens the Bug Reporting page.
3. QA Engineer enters bug details:
 - Bug title
 - Description
 - Severity
 - Priority
 - Screenshot
4. Frontend sends the bug information to the backend API.
5. Backend validates the information.
6. Backend stores the bug details in the database.
7. Database returns confirmation.
8. Backend sends success response to frontend.
9. System displays "Bug reported successfully" message.

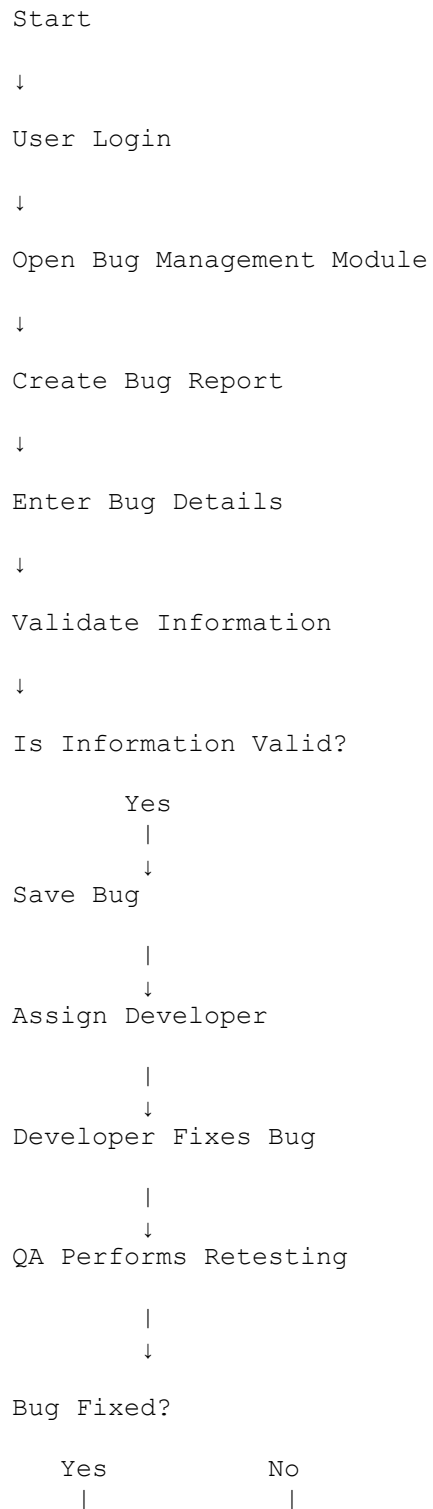
Flow:

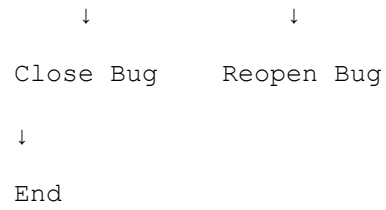


↓
Frontend
|
↓
QA Engineer

18. Activity Diagram

Activity: Bug Management Process





This activity flow represents the complete defect lifecycle from reporting to resolution.

19. Project Gantt Chart

The project will be developed over a period of 15 weeks. The following timeline represents the planned activities.

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Requirement Gathering	✓	✓													
Project Proposal Preparation		✓	✓												
System Analysis & Design			✓	✓											
Database Design			✓	✓											
UI/UX Design				✓	✓										
Authentication Module					✓	✓									
Project Management Module						✓	✓								
Requirement & Test Case Module							✓	✓							
Test Execution Module								✓	✓						
Bug Tracking Module									✓	✓					
Dashboard & Reports										✓	✓				
API Integration & Optimization											✓				
Manual Testing												✓			
Automation Testing													✓		
Documentation & Final Presentation														✓	✓

20. Risk Analysis

Risk Management

20. Risk Analysis

Risk analysis identifies possible challenges that may affect the successful completion of the project and defines strategies to reduce their impact.

Risk	Impact	Mitigation Strategy
Lack of technical knowledge	Medium	Research required technologies and follow documentation
Time management issues	High	Follow the planned timeline and complete tasks weekly
Database design problems	Medium	Perform proper analysis before implementation
Software bugs during development	Medium	Conduct regular testing and debugging
Requirement changes	Medium	Maintain flexible system design
Integration issues	Medium	Perform continuous testing between modules
Data security issues	High	Implement authentication and authorization mechanisms

Proper planning, continuous testing, and regular progress evaluation will help minimize project risks.

21. Testing Plan

Testing will be performed throughout the development process to ensure the quality, reliability, and functionality of the QA Management Suite.

Testing Types

1. Unit Testing

Individual system components will be tested separately to verify that each function works correctly.

Examples:

- Login validation
- Bug creation function
- Test case creation function

2. Integration Testing

Integration testing will verify communication between different system modules.

Examples:

- Frontend and backend communication
- Backend and database connection
- User authentication flow

3. System Testing

The complete system will be tested to ensure all requirements are satisfied.

Examples:

- Complete bug lifecycle testing
- Test execution workflow testing

4. User Acceptance Testing (UAT)

The system will be evaluated from the perspective of end users such as QA engineers and developers.

5. API Testing

REST APIs will be tested using Postman.

Testing includes:

- Request validation
- Response validation
- Error handling
- Authentication testing

6. Automation Testing

Selenium WebDriver will be used to automate important user workflows.

Automated scenarios:

- User login
- Create project
- Create test case
- Report bug
- Update bug status

7. Performance Testing

The system performance will be evaluated under multiple user requests.

Testing Documentation

The following documents will be maintained:

- Test Plan
- Test Scenarios
- Test Cases
- Test Execution Reports
- Bug Reports
- Test Summary Report

22. Sample Test Cases

Test cases are created to verify that each functionality of the QA Management Suite works according to the expected requirements.

Module: User Login

Test Case ID	Test Scenario	Test Steps	Expected Result
TC_LOGIN_001	Login with valid credentials	Enter valid email and password, click Login	User should successfully login and access the dashboard
TC_LOGIN_002	Login with invalid password	Enter correct email and incorrect password	System should display an error message
TC_LOGIN_003	Login with empty fields	Leave fields empty and click Login	System should display validation messages

Module: Bug Management

Test Case ID	Test Scenario	Test Steps	Expected Result
TC_BUG_001	Create new bug report	Enter bug details and submit	Bug should be created successfully
TC_BUG_002	Assign bug to developer	Select developer and assign bug	Bug should appear in developer dashboard
TC_BUG_003	Update bug status	Change status from Open to Fixed	Updated status should be displayed

Module: Test Case Management

Test Case ID	Test Scenario	Test Steps	Expected Result
TC_TEST_001	Create test case	Enter test case details and save	Test case should be stored successfully
TC_TEST_002	Execute test case	Select test result as Pass/Fail	Execution result should be recorded
TC_TEST_003	Edit test case	Update test case information	Changes should be saved

23. Bug Report Template

Bug Report

Bug ID: BUG-001

Title: Login button does not respond after entering valid credentials

Module: Authentication

Reported By: QA Engineer

Date Reported:

Severity: High

Priority: High

Environment:

- Browser:
- Operating System:
- Application Version:

Steps to Reproduce:

1. Open login page.
2. Enter valid username and password.
3. Click login button.

Expected Result:

User should be redirected to the dashboard.

Actual Result:

User remains on the login page.

Status:

Open

Assigned To:

Developer Name

Attachment:

24. Requirement Traceability Matrix (RTM)

The Requirement Traceability Matrix ensures that every system requirement is linked with related test cases.

Requirement ID	Requirement	Test Case ID	Status
R001	User Login	TC_LOGIN_001, TC_LOGIN_002	Completed
R002	Create Bug Report	TC_BUG_001	Completed
R003	Assign Bug	TC_BUG_002	Completed
R004	Execute Test Case	TC_TEST_002	Completed
R005	Generate Reports	TC_REPORT_001	Pending

25. Final Project Deliverables

The following deliverables will be provided after completing the project:

Software Deliverables

- Developed Web Application
- Source Code Repository
- Database Design
- Deployment Version

Documentation Deliverables

- Project Proposal
- Software Requirement Specification (SRS)
- System Design Documents
- Test Plan
- Test Cases
- Bug Reports
- Final Project Report

Testing Deliverables

- Postman API Collection
- Selenium Automation Scripts

- Test Execution Reports
 - Bug Tracking Records
 - Performance Testing Results
-

26. Project Summary

The QA Management Suite is a comprehensive web-based solution designed to improve software testing and defect management processes. The system provides a structured environment where QA engineers can manage test cases, execute tests, report defects, and monitor software quality.

By combining software development practices with quality assurance methodologies, this project demonstrates practical knowledge in full-stack development, database management, software testing, and system design. The solution can serve as a valuable tool for small development teams while also acting as a learning platform for software testing practices.